DECA Lab Logbook — Spring Term

Lab 3 | Task 4: Interrupt Mechanism

Task 4: Interrupt Mechanism (int-test1)

4.1 Program Overview

The eep1 sheet of lab3i is loaded with int-test1.ram. The base program enables interrupts with SETI and then loops, incrementing R0. The ISR at 0xFF00 services the INT1 timer interrupt.

4.2 Waveform Analysis: Changes Caused by Instructions vs Interrupt Mechanism

Simulate the eep1 sheet of lab3i running int-test1.ram in the waveform simulator. View IRQ1, IRQ1reset, NZCVout, and PCX in addition to PC, INS, R0, and R2. Trace execution from the moment IRQ1 is asserted through the interrupt jump, ISR execution, and the RETINT return to the base program. For each observed waveform change, determine whether it is caused by an instruction or by the interrupt mechanism.

```
SETI                    // enable interrupts
LOOP  ADD R0, #1
      JNE LOOP
      ORG 0xff00
ISR   STR R0, [0xFF80] // save base program R0
      MOV R0, Flags    // save base program flags
      STR R1, [0xFF81] // save base program R1
      MOV R1, 0b10
      STR R1, [0xFFFF] // Interrupt 1 reset high
      MOV R1, #0
      STR R1, [0xFFFF] // Interrupt 1 reset low
      LDR R1, [0xFF82]
      ADD R1, #1
      STR R1, [0xFF82]
      ADD R2, #1
      LDR R1, [0xFF81] // restore base program R1
      MOV Flags, R0   // restore base program flags
      LDR R0, [0xFF80] // restore base program R0
      RETINT
```

Changes caused by instructions:

- R0 increments: R0 increases by 1 every two clock cycles in the base program, driven by ADD R0, #1 (two cycles due to pipeline latency).
- IRQ1reset pulse: Generated by the two STR R1, [0xFFFF] instructions in the ISR — writing 0b10 then 0 to 0xFFFF creates a one-cycle high pulse on IRQ1reset.
- R2 increments: R2 increases by 1 approximately 10 cycles after IRQ1 is asserted, caused by ADD R2, #1 in the ISR.
- NZCVout changes: The flags are modified by arithmetic instructions during ISR execution, and are explicitly restored by MOV Flags, R0 near the end of the ISR.

Changes caused by the interrupt mechanism:

- IRQ1 asserted periodically: Caused by the external hardware timer counter, independent of the software.
- PC jumps to 0xFF00: When IRQ1 is asserted and FFIEN = 1, the interrupt hardware automatically redirects the PC to the fixed interrupt vector address 0xFF00.
- PCX updated: The hardware stores the return address (the PC value at the point of interruption) into the PCX register. In the waveform this is observed at approximately cycle 22, where PCX captures the current base-program PC.

Waveform simulation:

4.3 Why the Base Program PC, Flags, and R0 Remain Unaffected

Using the code and waveforms, explain why the base program PC, flags, and R0 remain unaffected by ISR execution.

PC: On interrupt entry, the hardware stores the base program return address into PCX. At the end of the ISR, RETINT restores PC from PCX, returning execution to the exact point where it was interrupted.

Flags: The ISR saves the current flags to R0 using MOV R0, Flags at the start, and restores them using MOV Flags, R0 before RETINT. The base program therefore sees the same NZCV state as before the interrupt.

R0: The ISR saves R0 to memory at 0xFF80 using STR R0, [0xFF80] at the start, and restores it using LDR R0, [0xFF80] before RETINT. The base program's R0 value is unaffected.